

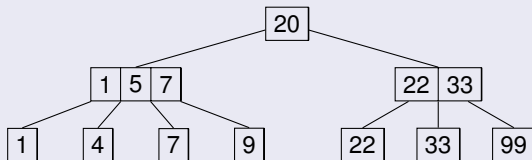
Definition

An (a, b) -tree is a search tree that satisfies the following properties:

- 1 The parameters a and b are integers such that $a \geq 2$ and $b \geq 2a - 1$.
- 2 All internal nodes, except for the root, must possess at least a and at most b children.
- 3 The root may have at most b children.
- 4 All leaves are required to be at the same depth.

Elements are stored exclusively in the leaves to simplify the explanation. ①

Example: (2,4)-Tree



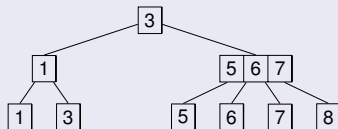
Operation FIND

The search operation commences at the root, utilizing the keys stored in the internal nodes.

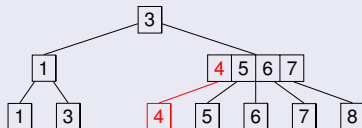
- 1 In the assignment, elements are stored in all nodes. This variant will be discussed in the tutorials.

(a,b)-tree: Operation INSERT

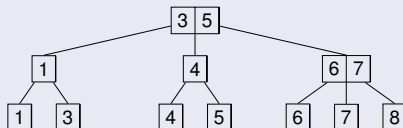
Insert 4 into the following (2,4)-tree



Add a new leaf into the proper parent

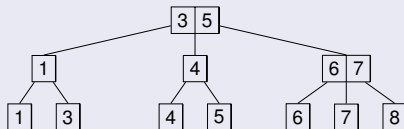


Recursively split node if needed

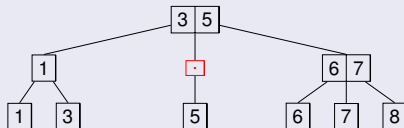


(a,b)-tree: Operation DELETE

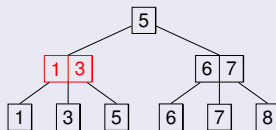
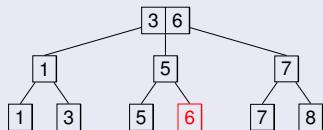
Delete 4 from the following (2,4)-tree



Find and delete the proper leaf



Recursively either share nodes from a sibling or fuse the parent



Elements Stored in Inner Nodes

- In the lecture, we stored data only in the leaves of the tree.
- In your assignment, data is stored in all inner nodes.
- Describe the operations for Insert and Delete in this context.
- Discuss applications where each version is preferred.

Using a Smaller Amount of Memory

- The number of children of a node is between a and b , where $b \geq 2a - 1$.
- If a node has only a children, approximately half of its memory may remain unused.
- Is it possible to modify the Insert and Delete operations so that at most $1/3$ of a node's memory is unused?
- Specifically, describe the operations in a variant of the (a, b) -tree where $a \approx \frac{2}{3}b$.

Build

- Given n elements, how fast can be build an (a,b)-tree?
- Can we build (a,b)-tree faster than $\mathcal{O}(n \log n)$?
- How fast can be build an (a,b)-tree when given elements are sorted?

Split

Given an (a,b)-tree and a key k , split the tree into two (a,b)-trees so that one contains all elements smaller than k and the other all larger elements.

Merge

Given two (a,b)-tree such that all elements in the first one are smaller than all elements in the second one, merge these trees into one (a,b)-tree.

Goal

To achieve efficient parallelization of the operations FIND, INSERT, and DELETE, under the assumption that $b \geq 2a$.

Operation INSERT

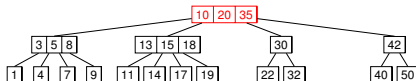
Split every node with b children along the path from the root to the inserted leaf.

Operation DELETE

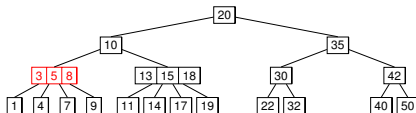
Update (either move a child or merge with a sibling) every node with a children along the path from the root to the deleted leaf.

(a,b)-Tree: Parallel Access: Example

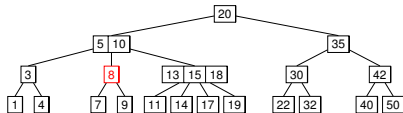
- Insert an element with key 6 into the following (2,4)-tree:



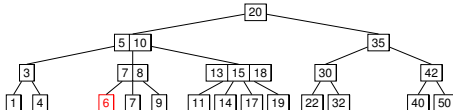
- First, we split the root:



- Next, we proceed to the left child, which we also split:



- Finally, the node with key 8 does not require a split, allowing for the addition of a new node:



Example of Sizes and Speeds of Different Types of Memory

	Size	Speed
L1 cache	32 KB	223 GB/s
L2 cache	256 KB	96 GB/s
L3 cache	8 MB	62 GB/s
RAM	32 GB	23 GB/s
SSD	112 GB	448 MB/s
HDD	2 TB	112 MB/s

Trivial Program

```
# Initialize an array of 32-bit integers of size  $n$ 
1 for ( $i=0; i+d < n; i+=d$ ) do
2    $A[i] = i+d$  # Take every  $d$ -th position and create a loop
3  $A[i]=0, i=0$ 
# Measure the duration of the loop depending on  $n$  and  $d$ 
# The number of operations is independent of  $n$  and  $d$ 
4 for ( $j=0; j < 2^{28}; j++$ ) do
5    $i = A[i]$  # Repeatedly traverse the loop of  $d$ -th positions
```